

# Quantum compilation and resource estimation

## (I) Quantum compilation overview

Olivia Di Matteo (UBC)

AlgoLab Summer School – Monday 15 June 2026



THE UNIVERSITY OF BRITISH COLUMBIA  
Electrical and Computer Engineering  
Faculty of Applied Science



Stewart Blusson  
Quantum Matter Institute  
THE UNIVERSITY OF BRITISH COLUMBIA



CANADA  
RESEARCH CHAIRS  
CHAIRES DE  
RECHERCHE DU  
CANADA



**NSERC**  
**CRSNG**

# Expectations vs. reality

really?

“Quantum computers will be able to solve

~~every~~ problem exponentially faster.”

Some

not always  
exponential

??

# Expectations vs. reality

in the future, at scale, are anticipated to

*“Quantum computers will be able to solve*

*every problem exponentially faster.”*

some classes of problems      with theoretical scaling advantages  
(that are exponential in some cases)

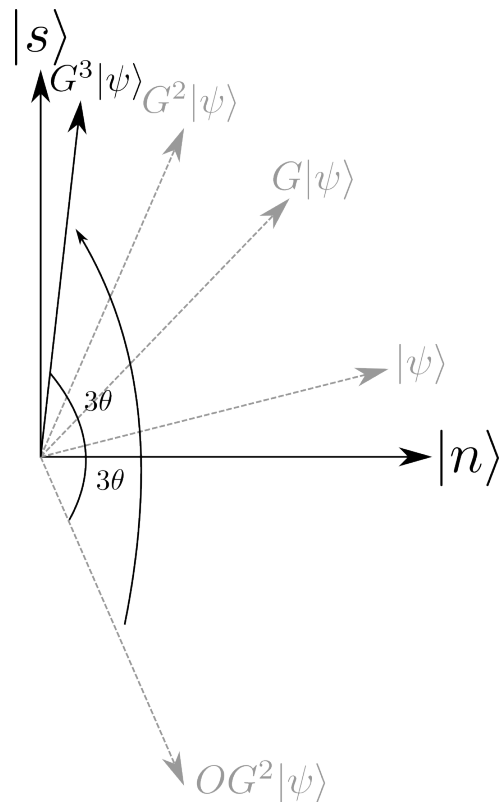
# Gotta go fast!

Grover's algorithm is one of the few quantum algorithms with a **provable quantum speed up**.

Oracle queries to find a marked item in an  $N$ -bit search space:

$O(2^N)$   
classical

$O(\sqrt{2^N})$   
quantum



# Question

How much does it cost to run a preimage attack on the **SHA2-256** hash function with Grover search on a fault-tolerant quantum computer?

# Question

How much does it cost to run a preimage attack on the **SHA2-256** hash function with Grover search on a fault-tolerant quantum computer?

(A)  $2^{128}$

4

(B)  $2^{145}$

(C)  $2^{166}$

(D)  $2^{127.6}$

# Question

How much does it cost to run a preimage attack on the **SHA2-256** hash function with Grover search on a fault-tolerant quantum computer?

(A)  $2^{128}$

(B)  $2^{145}$

(C)  $2^{166}$

(D)  $2^{127.6}$

(E) *This question can't be answered meaningfully until you define "cost", specify an error correcting code and associated computational model, and state your assumptions about hardware noise.*

# Resource estimation is *important*



arXiv > quant-ph > arXiv:2505.15917

Search... Help | Adv

**Quantum Physics**

[Submitted on 21 May 2025]

**How to factor 2048 bit RSA integers with less than a million noisy qubits**

Craig Gidney



arXiv > quant-ph > arXiv:2603.28627

Search... Help | Adv

**Quantum Physics**

[Submitted on 30 Mar 2026]

**Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits**

Madelyn Cain, Qian Xu, Robbie King, Lewis R. B. Picard, Harry Levine, Manuel Endres, John Preskill, Hsin-Yuan Huang, Dolev Bluvstein



# Resource estimation is *important*

arXiv > quant-ph > arXiv:2505.15917 Search... Help | Adv

**Quantum Physics**

[Submitted on 21 May 2025]

## How to factor 2048 bit RSA integers with less than a million noisy qubits

[Craig Gidney](#)

**GIZNEWS DAILY**

### One-Million-Qubit Machine Can Break 2048-Bit RSA in 7 Days



Written by [GizNews](#) in [All News](#), [Computers](#) May 27, 2025

arXiv > quant-ph > arXiv:2603.28627 Search... Help | A

**Quantum Physics**

[Submitted on 30 Mar 2026]

## Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits

[Madelyn Cain](#), [Qian Xu](#), [Robbie King](#), [Lewis R. B. Picard](#), [Harry Levine](#), [Manuel Endres](#), [John Preskill](#), [Hsin-Yuan Huang](#), [Dolev Bluvstein](#)

ScienceNews

**NEWS** QUANTUM PHYSICS

## Just 10,000 quantum bits might crack internet encryption schemes

Computers based on atoms could demand a fraction of the qubits previously estimated

**PHYSICS & CHEMISTRY**

## How Should We Prepare for the Looming Quantum Encryption Apocalypse?

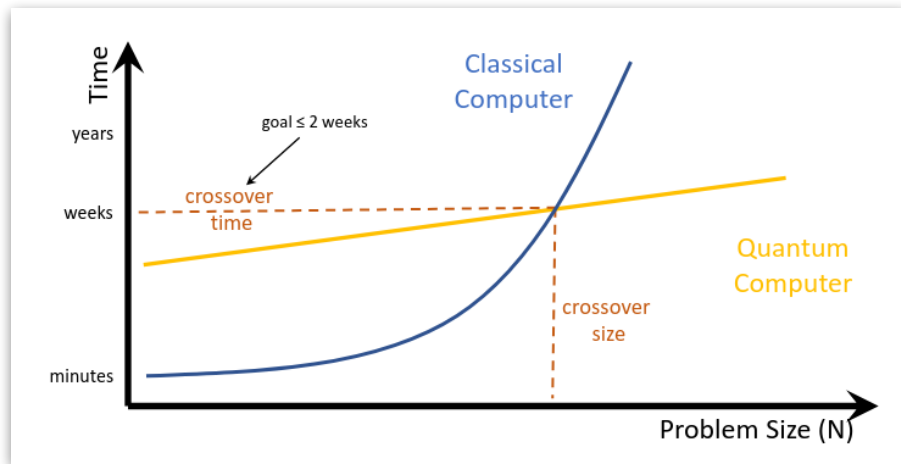
The dreaded Q-day could arrive sooner than expected, and when it does, experts say we need to be ready.

BY GAYOUNG LEE  
PUBLISHED APRIL 11, 2026, 7:00 AM ET

# Quantum resource estimation

How much does it *really* cost to run a quantum algorithm?

- Qubits?
- Gates?
- Time?
- Energy?
- Money?



*Speed vs. scaling*

*How to answer these questions?*

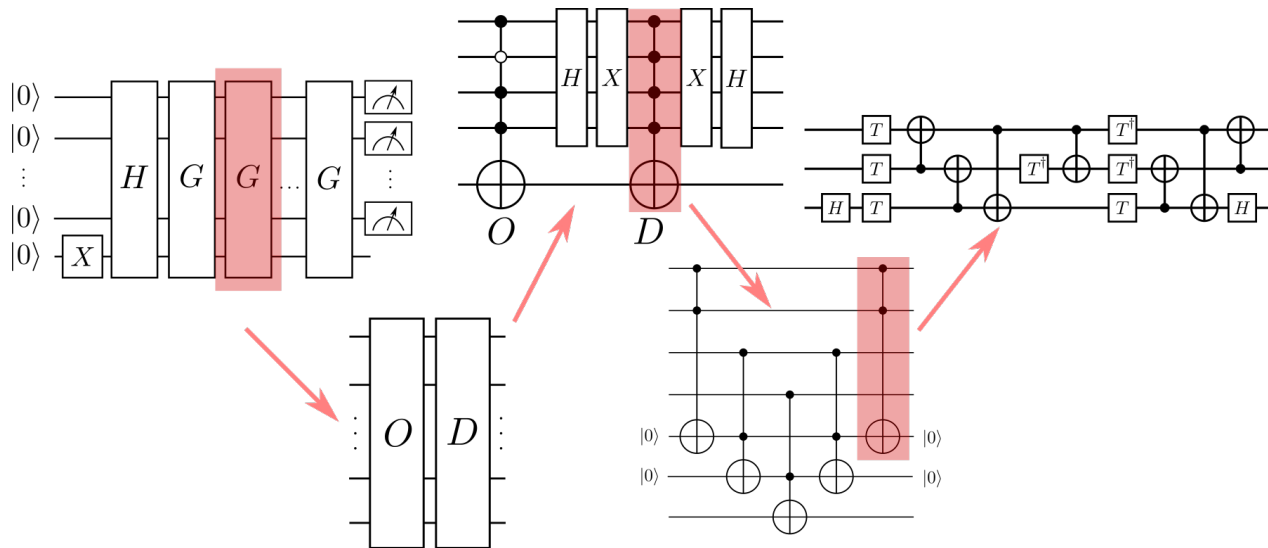
T. Hoefler, T. Häner, and M. Troyer (2023) Disentangling Hype from Practicality: On Realistically Achieving Quantum Advantage. arXiv:2307.00523 [quant-ph].

# Quantum compilation and resource estimation

## Learning outcomes:

1. List and describe the steps of the quantum compilation pipeline
2. Apply compilation techniques in software and perform logical-level resource estimates
3. Estimate the physical resources required to run an algorithm with error correction

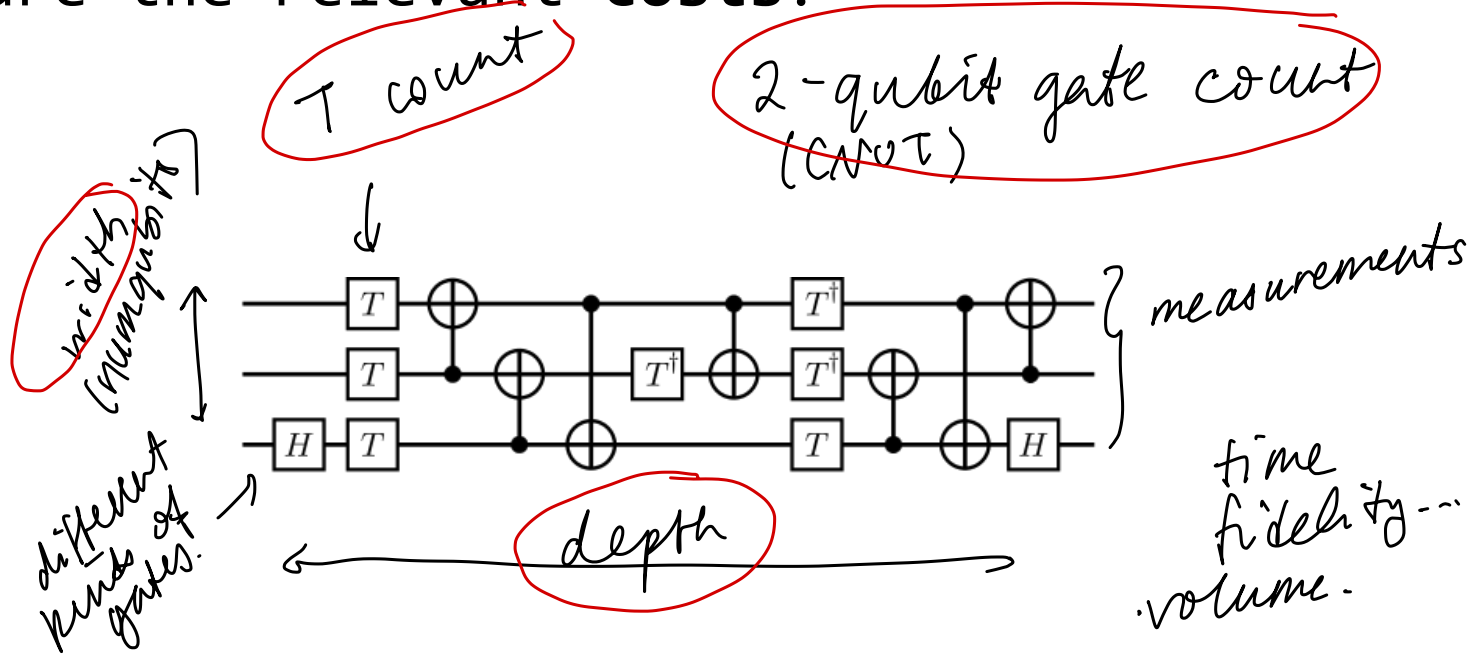
# Lecture 1 in one slide



How do we do this in the best way possible? What defines “best”?

# What are the relevant costs?

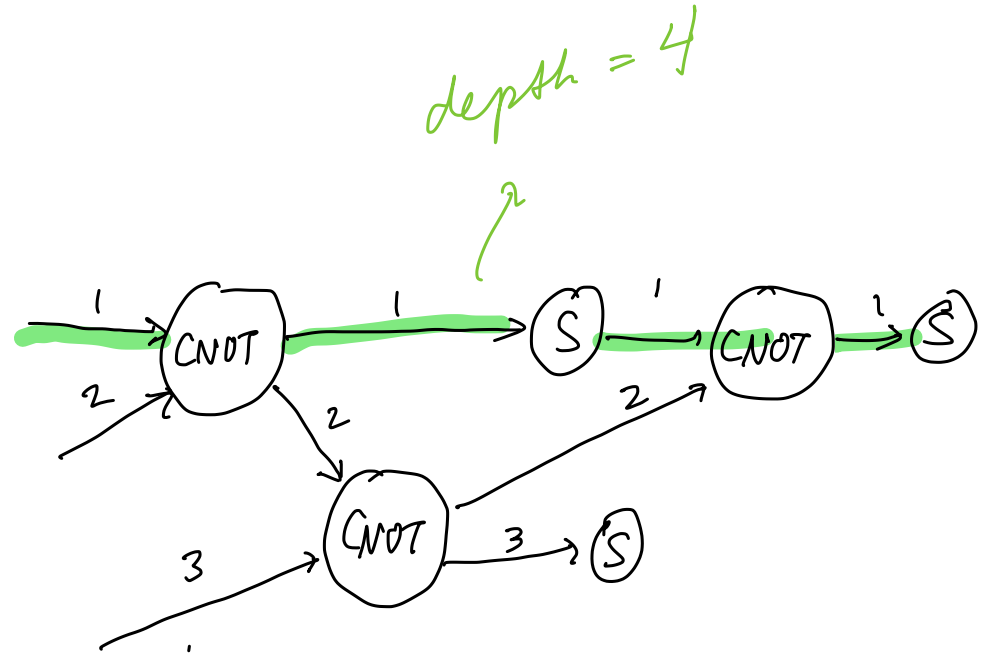
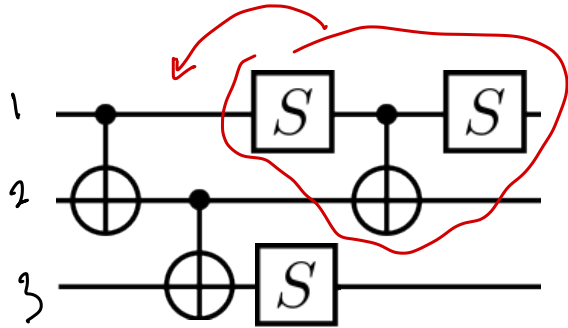
1-qubit also



width: 3      T-count: 7      2-qubit gates = 7  
1-qubit gates: 9      depth: 11

# Circuit depth

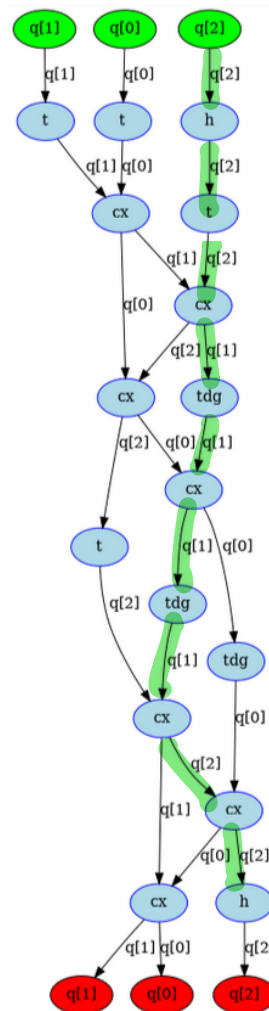
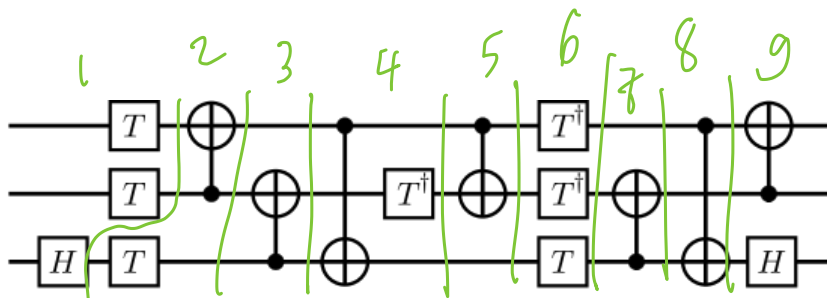
The circuit depth is the length of the longest path in its **directed acyclic graph (DAG)** representation.



# Circuit depth

Generally don't do by hand;  
NetworkX is your friend.

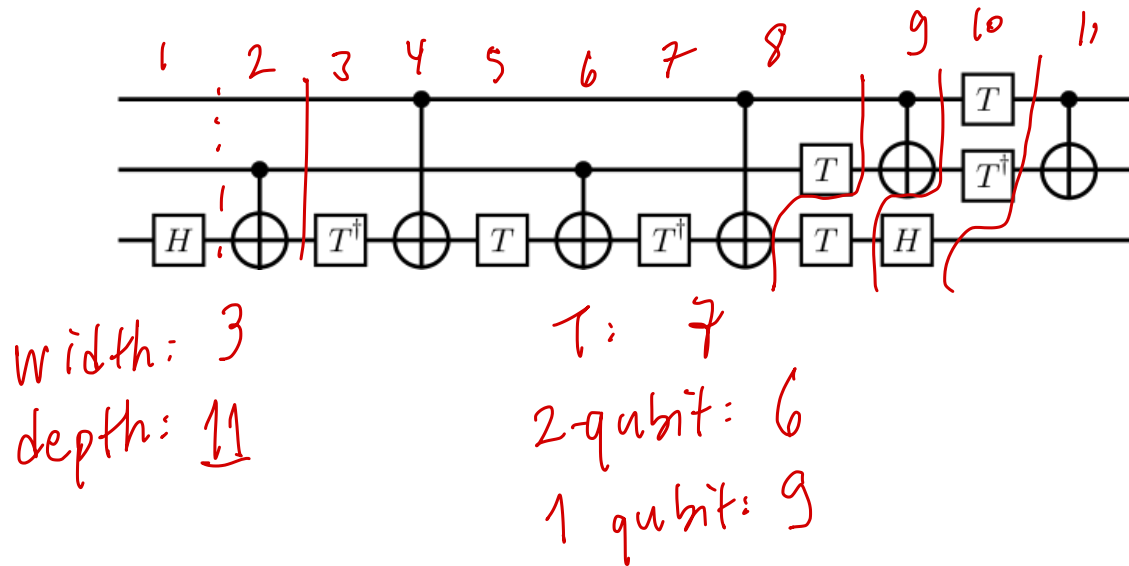
<https://networkx.org/>



(DAG generated  
in Qiskit)

# Exercise

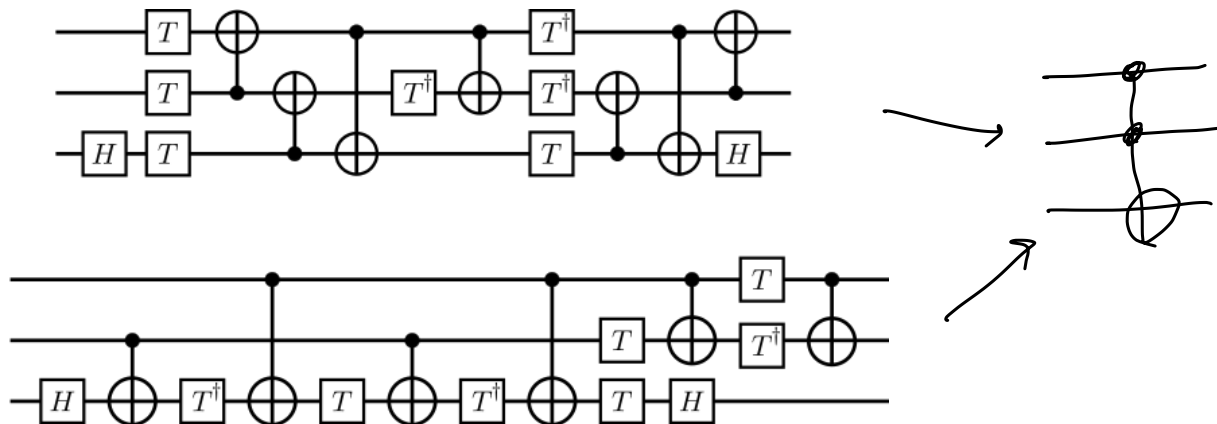
Determine depth, width, and gate counts of the following circuit.





# Question

What do these circuits do?



US

High-level algorithmic description

Software  
(Human-readable code)

CPU

# Compilers are essential

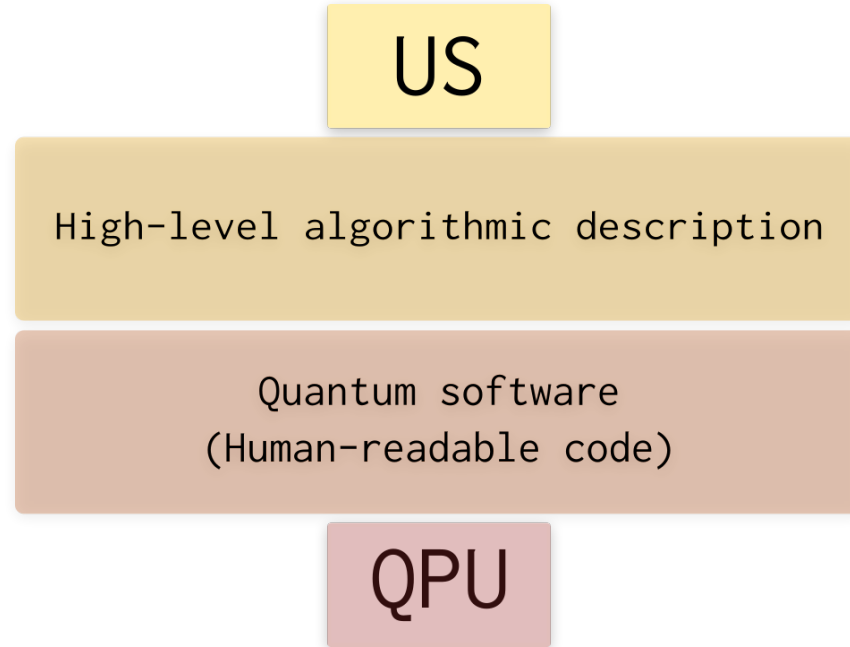
```
#include<stdio.h>

void main() {
    printf("Hello, world!");
}
```

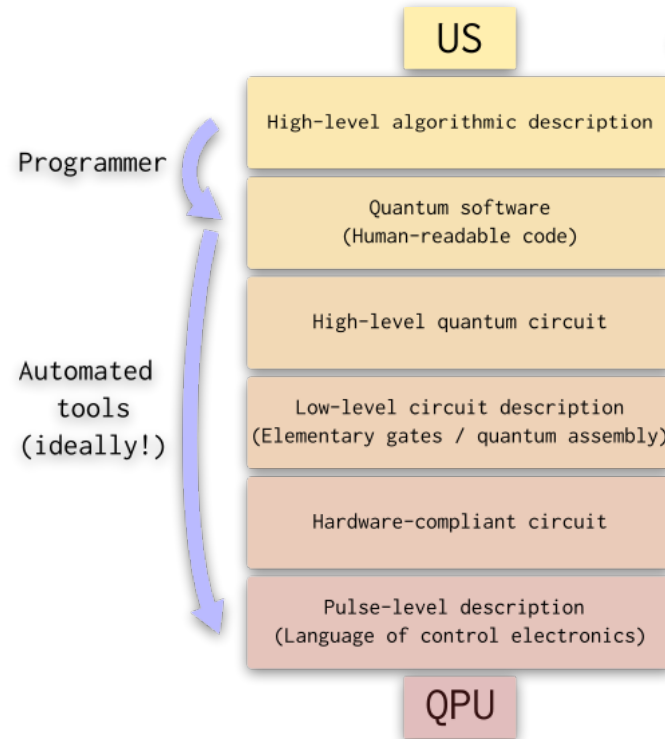
```
.file "hello_world.c"
.text
.section .rodata
.LC0:
.string "Hello, world!"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
endbr64
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
leaq .LC0(%rip), %rdi
movl $0, %eax
call printf@PLT
nop
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (Ubuntu 9.2.1-9ubuntu2) 9"
```

```
11a0 ff48c1fd 03741f31 db0f1f80 00000000 .H...t.1.....
11b0 4c89f24c 89ee4489 e741ff14 df4883c3 L..L..D..A...H..
11c0 014839dd 75ea4883 c4085b5d 415c415d .H9.u.H...[JA\A]
11d0 415e415f c366662e 0f1f8400 00000000 A^A_.ff.....
11e0 f30f1efa c3
Contents of section .fini:
11e8 f30f1efa 4883ec08 4883c408 c3 ....H...H....
Contents of section .rodata:
2000 01000200 48656c6c 6f2c2077 6f726c64 ....Hello, world
2010 2100
Contents of section .eh_frame_hdr:
2014 011b033b 40000000 07000000 0cf0ffff ...;@.....
2024 74000000 2cf0ffff 9c000000 3cf0ffff t...,.....<...
2034 b4000000 4cf0ffff 5c000000 35f1ffff ....L...\...5...
2044 cc000000 5cf1ffff ec000000 ccf1ffff ....\.....
2054 34010000
Contents of section .eh_frame:
2058 14000000 00000000 017a5200 01781001 .....zR...x...
```

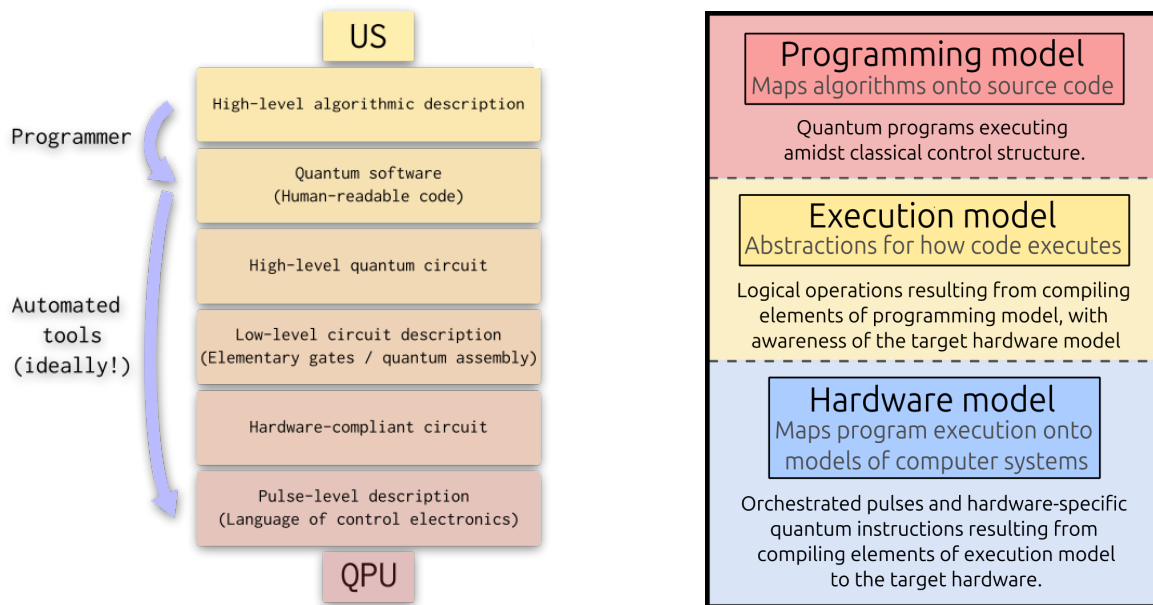
In an ideal world...



# High-level overview: quantum compilation stack



# Quantum compilation and abstraction



ODM, S. Núñez-Corrales, M. Stęchły, S. Reinhardt, and T. Mattson. 2024. An abstraction hierarchy towards productive quantum programming. In Proc. of 2024 IEEE International Conference on Quantum Computing and Engineering (QCE), Montreal, PQ. pp. 979-989.

# Quantum circuit\* compilation

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

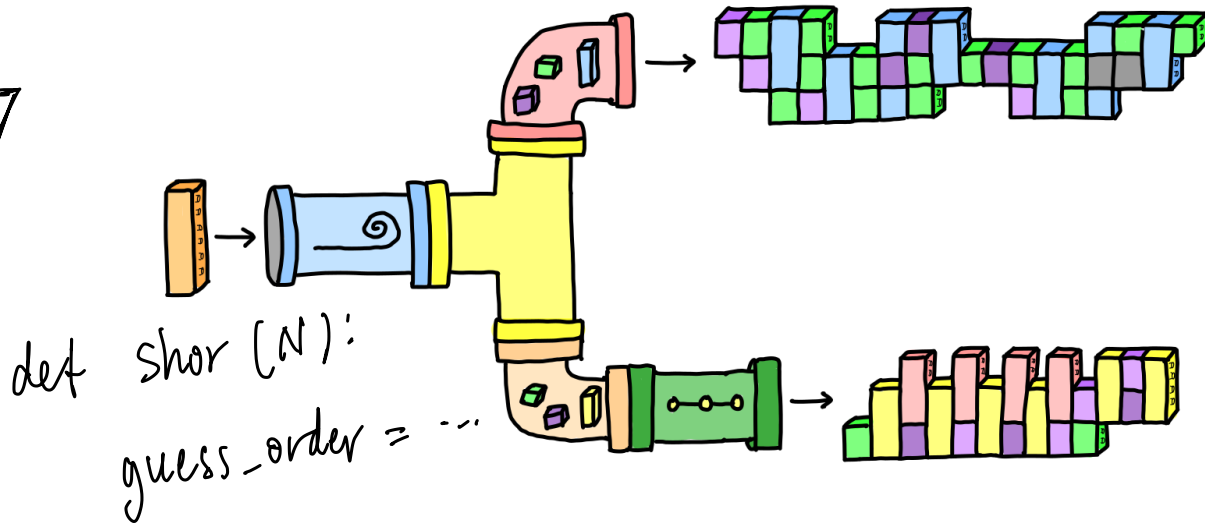
Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

Implemented as a sequence of modular *passes*, or *transforms*, that modify a circuit in various ways.



# Quantum circuit compilation

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

Implemented as a sequence of modular *passes*, or *transforms*, that modify a circuit in various ways.

Loosely divided into 3 stages:

- decomposition and synthesis
- hardware-independent optimization
- hardware-dependent optimization



# Circuit synthesis

Given:

- arbitrary unitary matrix  $U$
- a finite set of gates  $\{G_k\}$   $CNOT, H, T$
- a target precision,  $\epsilon$

Find

$$U' = C_1 C_2 \cdots C_m$$

s.t.

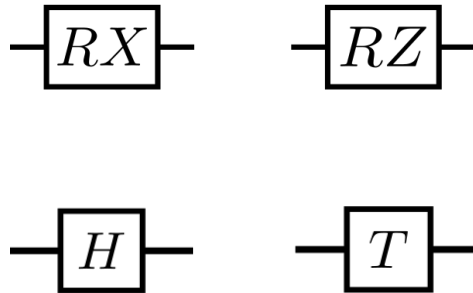
$$\|U - U'\| < \epsilon, C_i \in \{G_k\}$$

# Universal gate sets

A  $\{G_k\}$  for which this can be done is called a **universal gate set**.

Examples for a single qubit:

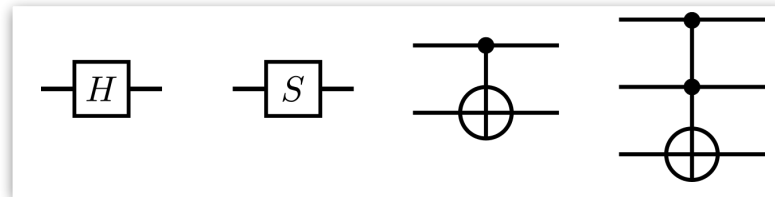
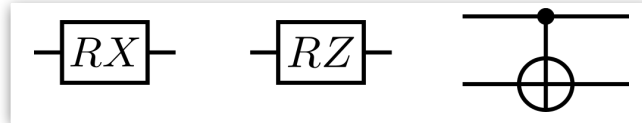
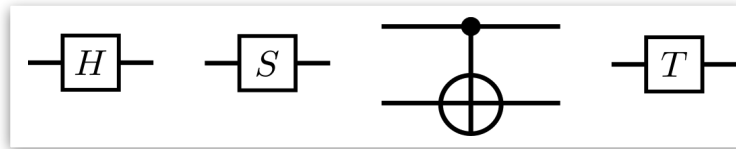
- Any two Pauli rotations (exact)
- Hadamard and  $T$  (approximate)



Theoretical bound:  $O(\log^c(1/\epsilon))$  gates,  $c$  varies from 1-4 depending on assumptions.

# Universal gate sets

## Multi-qubit gate sets



## Exercise

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$$

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Express Pauli  $X$  in terms of  $H$  and  $T$ .

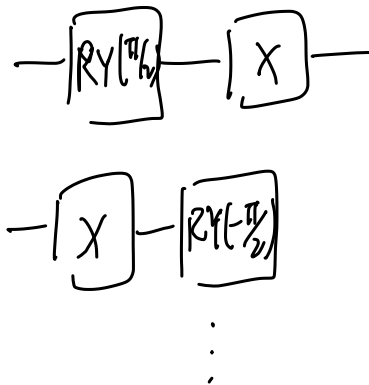
*Hint:* start by expressing it using  $H$  and  $Z$ .

$$\begin{aligned} X &= H Z H \\ &= H T T T T H \end{aligned}$$

# Exercise

Express Hadamard in terms of  $\{RX, RY, RZ\}$

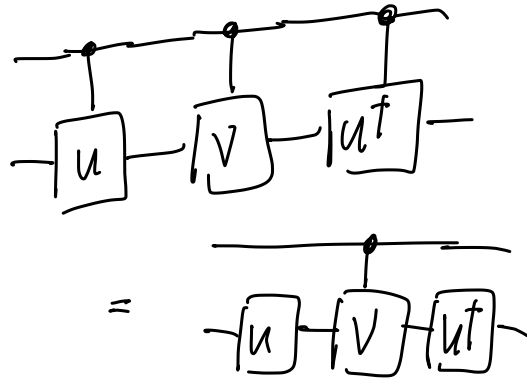
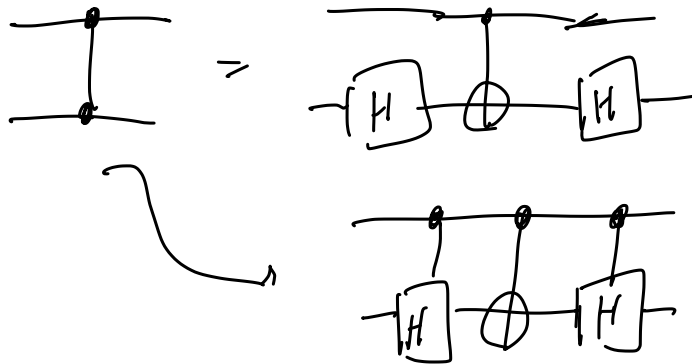
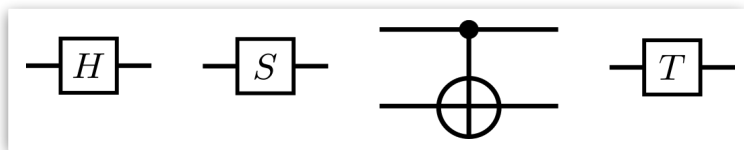
*Hint:* there are many ways to do this!



# Exercise

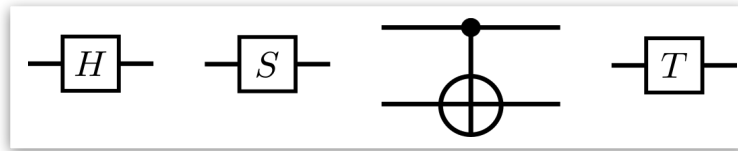
$$\mathcal{Z} = HXH$$

Express CZ using gates from



# Take-home exercises

1. Express controlled Hadamard using gates from



2. Express  $RZ(\pi/3)$  using Hadamard and  $T$  up to precision  $10^{-5}$ .

# Synthesis and decomposition techniques

For under-specified or arbitrary operations, or for special gate sets, we may need to *find* a decomposition.

- search problem
- number-theoretic techniques
- numerical methods

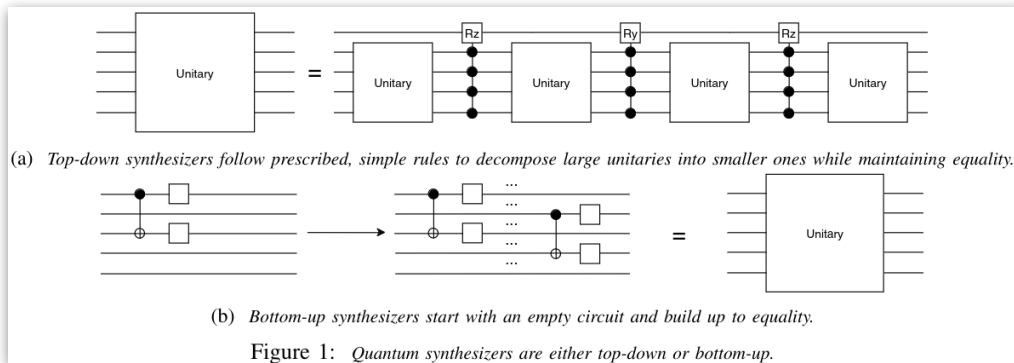


Image: E. Younis, K. Sen, K. Yelick, and C. Iancu. *QFAST: Conflating Search and Numerical Optimization for Scalable Quantum Circuit Synthesis*. arXiv:2103.07093 [quant-ph]



# Challenges

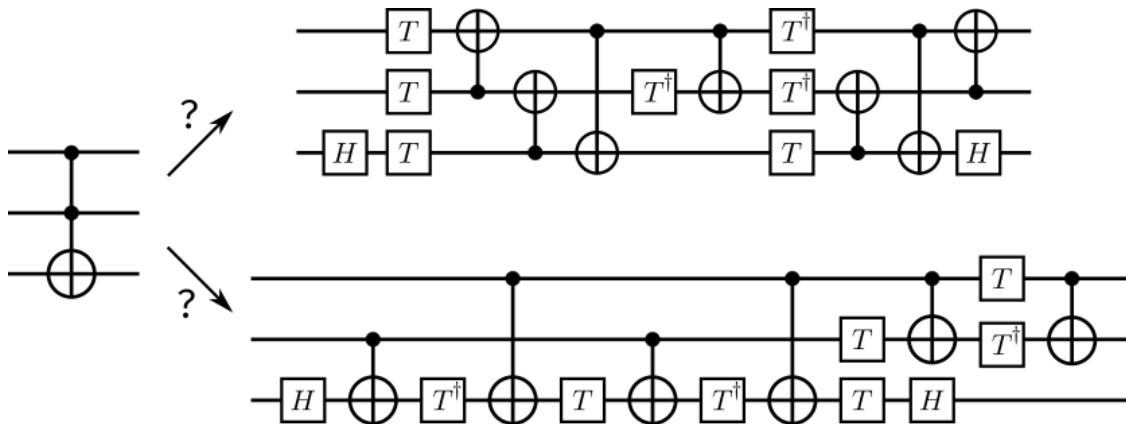
Synthesis is hard!

- algorithm complexity and computational resources

# Challenges

Synthesis is hard!

- algorithm complexity and computational resources
- how to choose the best decomposition to *enable further optimization* down the line?



# Challenges

Synthesis is hard!

- algorithm complexity and computational resources
- how to choose the best decomposition to *enable further optimization* down the line?
- manage tradeoffs between various resources

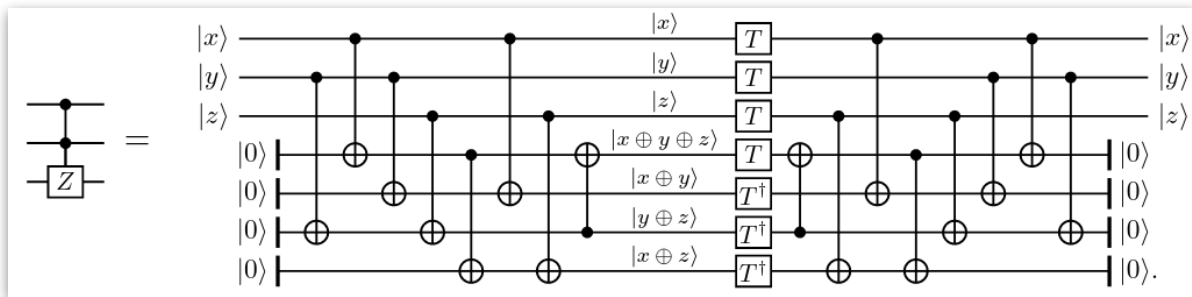
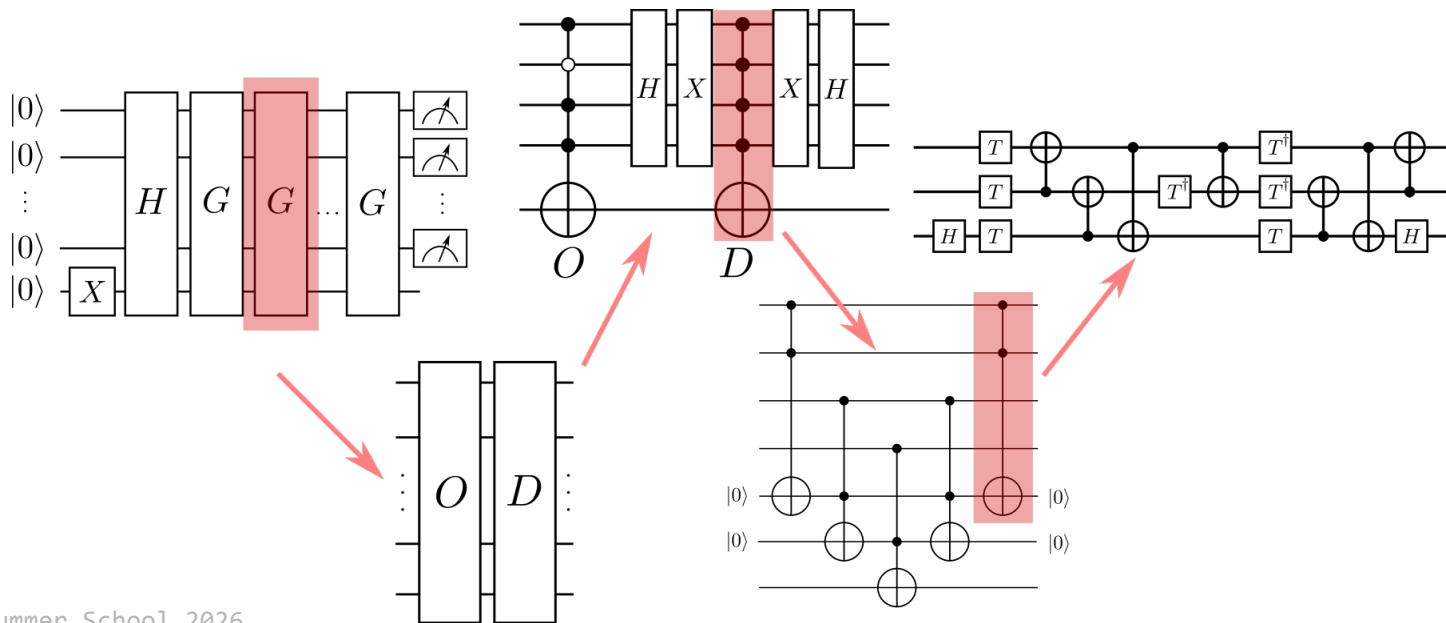


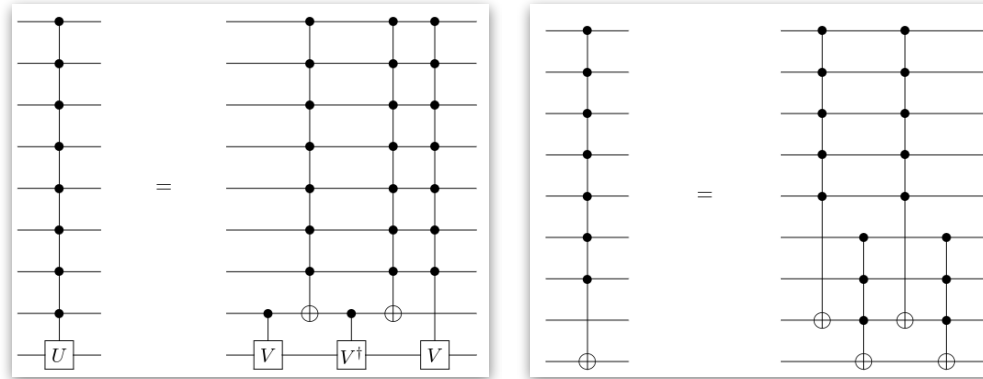
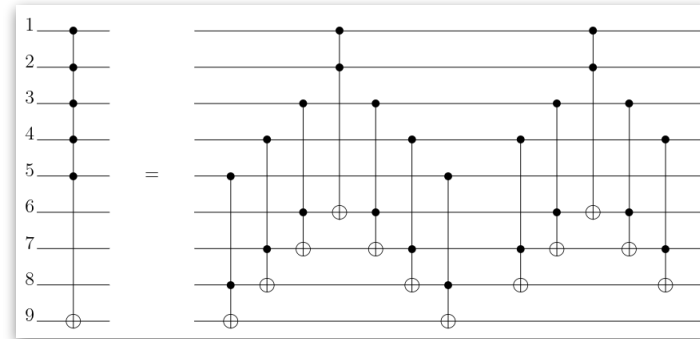
Image: P. Selinger (2013) *Quantum circuits of T -depth one*. Phys. Rev. A 87, 042302 (2013)

# Circuit decomposition

Decompositions of many common operations are **known**. Some are even straightforward to work out by hand.



# Multi-controlled NOT: space-time tradeoffs



Images: Barenco et al., *Elementary gates for quantum computation* Phys.Rev. A 52 (1995) 3457

# Quantum circuit compilation

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

Implemented as a sequence of modular *passes*, or *transforms*, that modify a circuit in various ways.

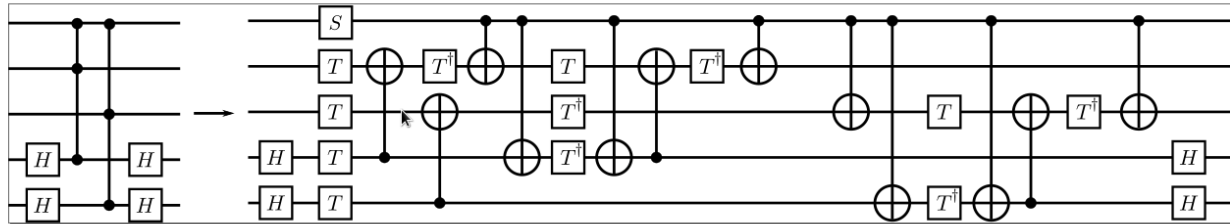
Loosely divided into 3 stages:

- decomposition and synthesis
- hardware-independent optimization
- hardware-dependent optimization

# Circuit optimization

Machine-independent (or dependent) sequence of *passes* to reduce:

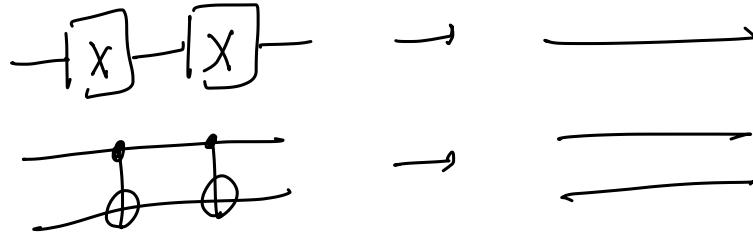
- gate count (overall, or for particular gate)
- circuit depth
- qubit count



# Optimization passes

Passes have varying levels of complexity:

- inverse cancellation





# Optimization passes

Passes have varying levels of complexity:

- inverse cancellation
- rotation merging

$$\text{---} \boxed{\text{RZ}(a)} \text{---} \boxed{\text{RZ}(b)} \text{---} = \text{---} \boxed{\text{RZ}(a+b)} \text{---}$$

# Optimization passes

Passes have varying levels of complexity:

- inverse cancellation
- rotation merging
- template matching

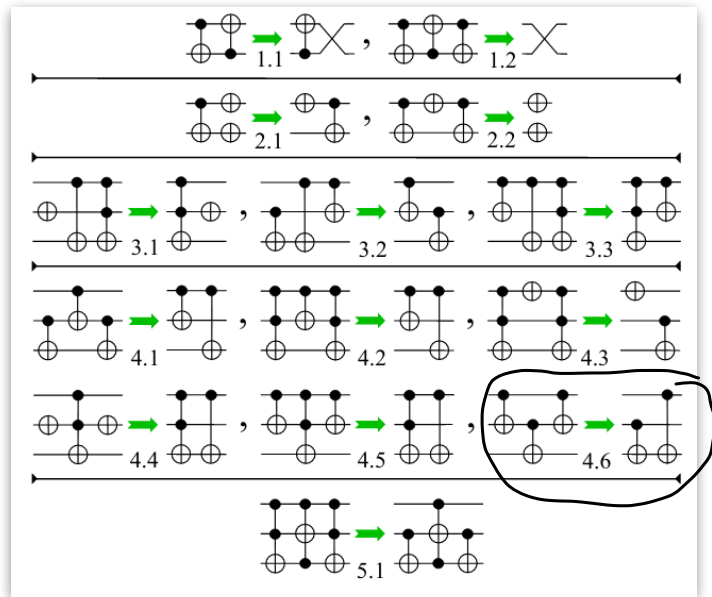
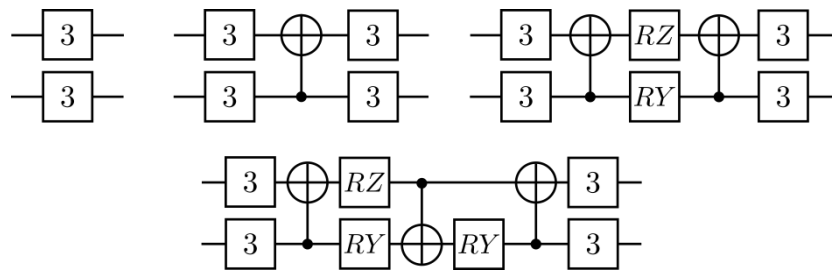


Image: D. M. Miller, D. Maslov, G. W. Dueck. *A Transformation Based Algorithm for Reversible Logic synthesis*. DAC 2003.

# Optimization passes

Passes have varying levels of complexity:

- inverse cancellation
- rotation merging
- template matching
- two-qubit block *resynthesis*

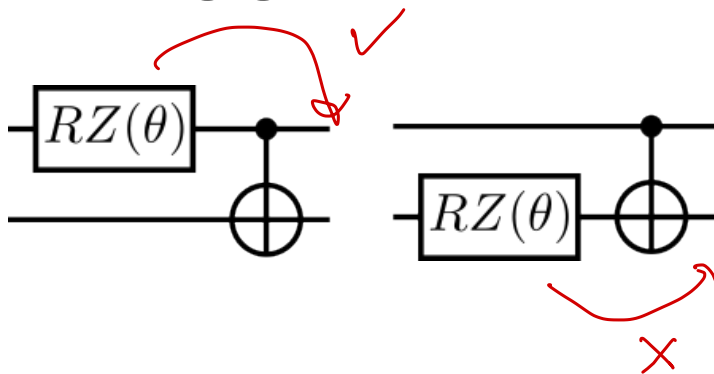


Reference: V. V. Shende, I. L. Markov, and S. S. Bullock (2004) *Minimal universal two-qubit controlled-NOT-based circuits*. Phys. Rev. A 69 062321.

# Commutation

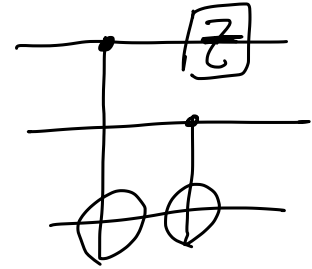
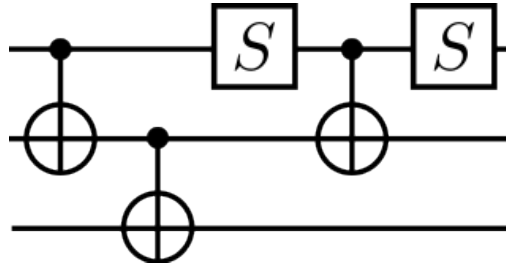
Commutation relations can help us move gates “through” each other to enable optimizations.

**Exercise:** do the following gates commute?



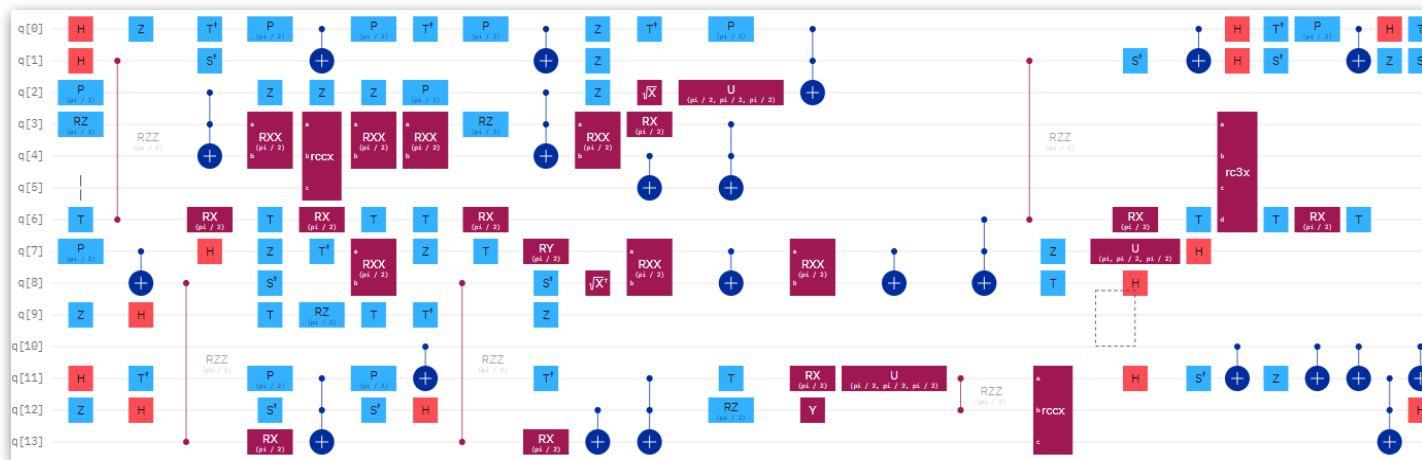
## Exercise: easy mode

Determine this circuit's resource counts, then optimize it.  
What is the minimum depth?



# Take-home exercise: hard mode

Determine this circuit's resource counts, then optimize it.  
What is the minimum depth?



Circuit credit: Mushahid Khan

# Quantum circuit compilation

US

High-level algorithmic description

Quantum software  
(Human-readable code)

High-level quantum circuit

Low-level circuit description  
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description  
(Language of control electronics)

QPU

Implemented as a sequence of modular *passes*, or *transforms*, that modify a circuit in various ways.

Loosely,

```
new_circuit = compiler_pass(old_circuit)
```

# Software for quantum compilation

Most general-purpose QC frameworks include compilation modules. There are also libraries specializing in compilation.

```
pass_manager = generate_preset_pass_manager(  
    optimization_level=1,  
    backend=backend,  
    layout_method="trivial",  
    routing_method="sabre",  
)  
  
compiled_circuit = pass_manager.run(circuit)
```



Qiskit

```
pipeline = qp.CompilePipeline(  
    qp.transforms.commute_controlled,  
    qp.transforms.cancel_inverses,  
    qp.transforms.merge_rotations,  
)  
  
compiled_qnode = pipeline(grover_qnode)
```



PENNYLANE

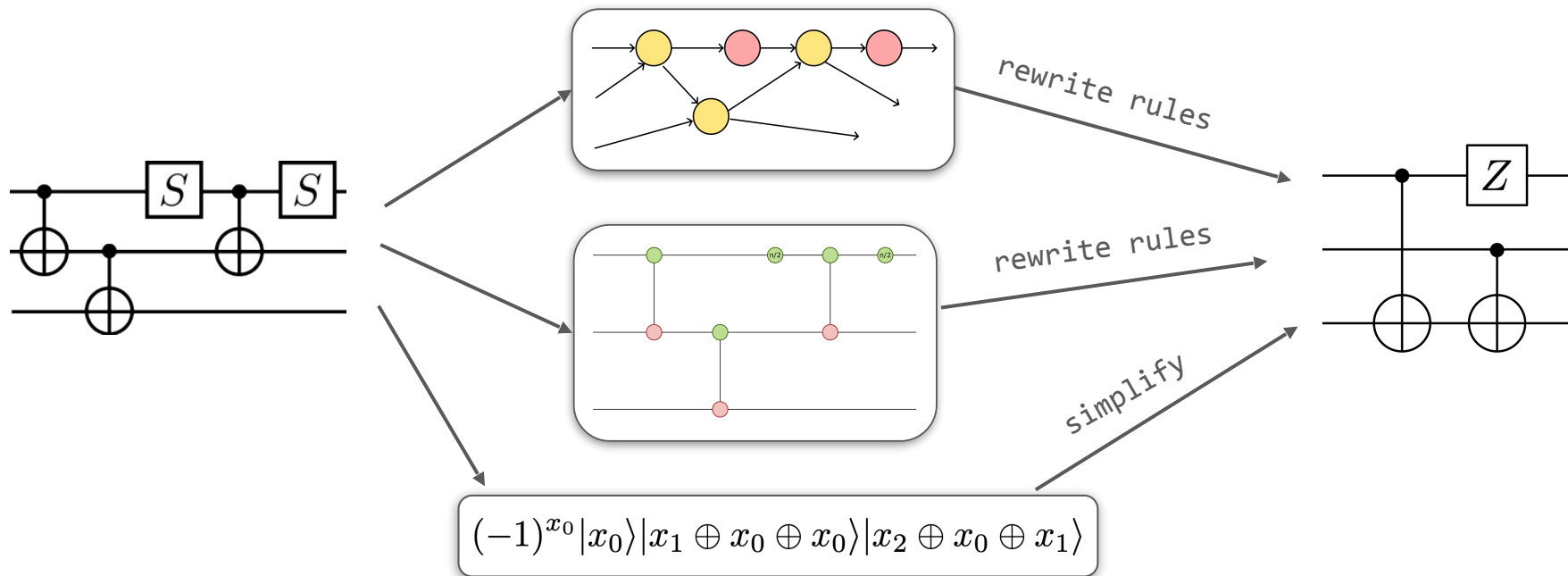
```
seqpass = SequencePass([  
    CommuteThroughMultis(),  
    RemoveRedundancies()  
)  
reppass = RepeatPass(seqpass)  
  
cu = CompilationUnit(circ)  
seqpass.apply(cu)  
compiled_circuit = cu.circuit
```

TKET



# Software for quantum compilation

Pipelines may compose passes acting on different representations.



# Software for quantum compilation

Some frameworks contain expansive libraries of passes. Determining a suitable *pass ordering* is an **open problem**.

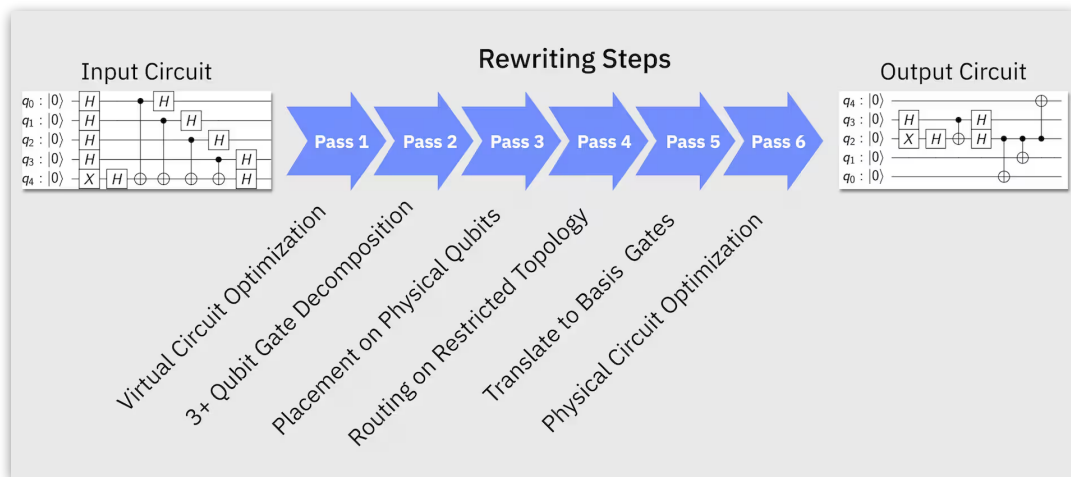
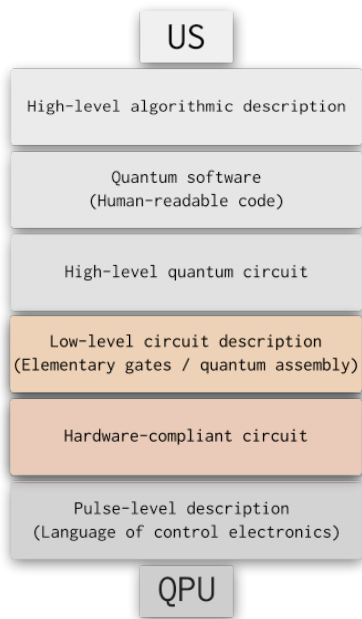


Image: <https://quantum.cloud.ibm.com/docs/en/api/qiskit/transpiler#overview>

```
passes.RoundAngles()  
passes.RoutingPass()  
passes.SimplifyInitial()  
passes.SimplifyMeasured()  
passes.SquashCustom()  
passes.SquashRzPhasedX()  
passes.SquashTK1()  
passes.SynthesiseTK()  
passes.SynthesiseTket()  
passes.ThreeQubitSquash()  
passes.ZXGraphlikeOptimisation()  
passes.ZZPhaseToRz()
```

pyTKET passes

# Quantum circuit compilation



Implemented as a sequence of modular *passes*, or *transforms*, that modify a circuit in various ways.

Loosely divided into 3 stages:

- decomposition and synthesis
- hardware-independent optimization
- hardware-dependent optimization

# Hardware consideration: gate set

Using a different gate set is like *speaking a different language*.



I would like to use a real  
quantum computer.

*I would like to use a real  
quantum computer.*



J'aimerais utiliser un vrai  
ordinateur quantique.

*I would like to use a real  
computer that is quantum.*

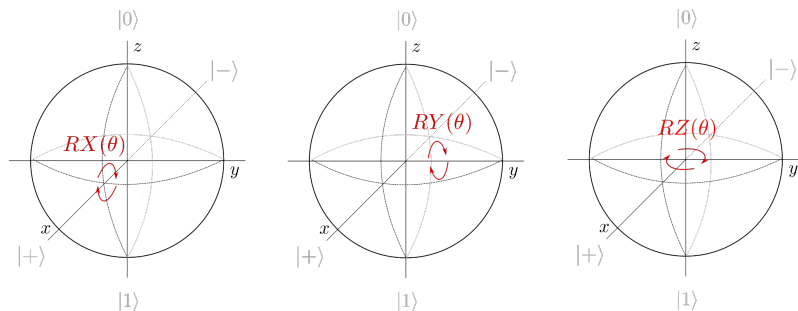


本当の量子コンピュータを使いたい。

*A real quantum computer, I  
would like to use.*

# Hardware consideration: gate set

$RX/RY/RZ$  are **arbitrary-angle** rotations about **fixed axes**



IonQ's trapped ion hardware native gates:  $GPI/GPI2$ , **fixed-angle** rotations about **arbitrary axes** in  $xy$ -plane

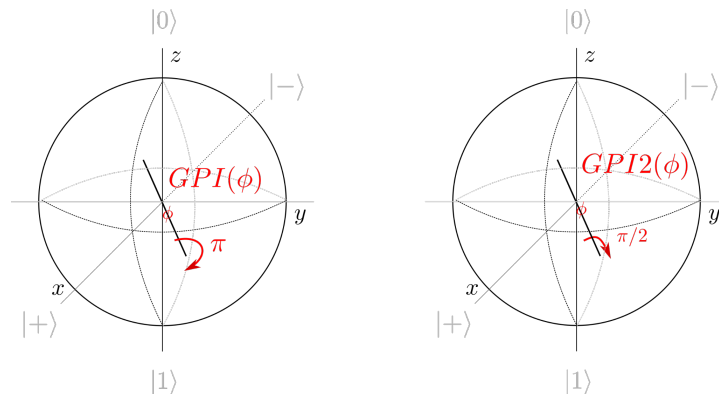


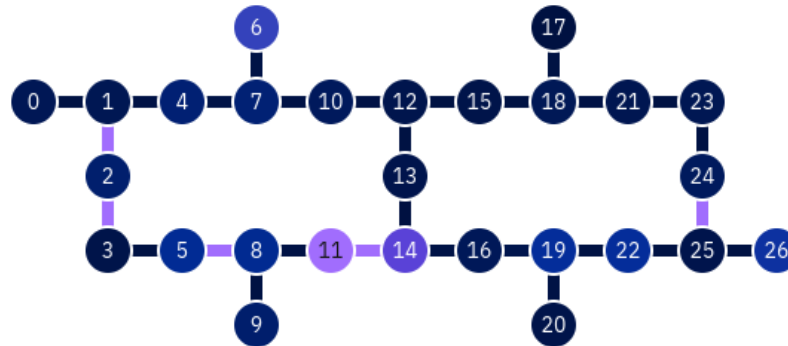
Image adapted from Xanadu Quantum Codebook Node I.6

See blog post: <https://pennylane.ai/blog/2023/06/the-ionizer-building-a-hardware-specific-transpiler-using-pennylane>

# Qubit placement and routing

Each type of machine has its own compilation challenges:

- superconducting machines have restricted topology

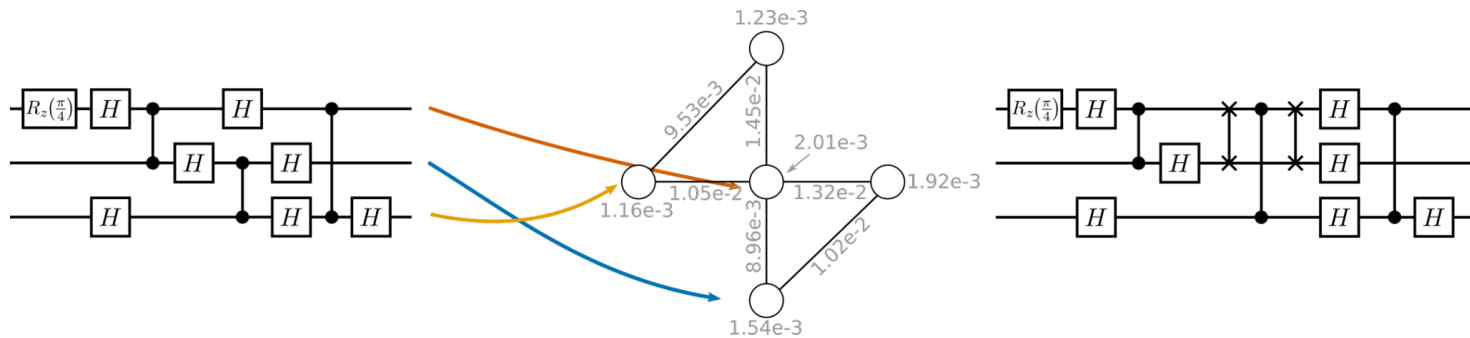


IBM Q Kolkata – screenshot 2023-08-24

# Qubit placement and routing

Each type of machine has its own compilation challenges:

- superconducting machines have restricted topology



Placement is NP-complete; routing is NP hard.

# Qubit placement and routing

Each type of machine has its own compilation challenges:

- superconducting machines have restricted topology
- neutral atom machines can lose atoms

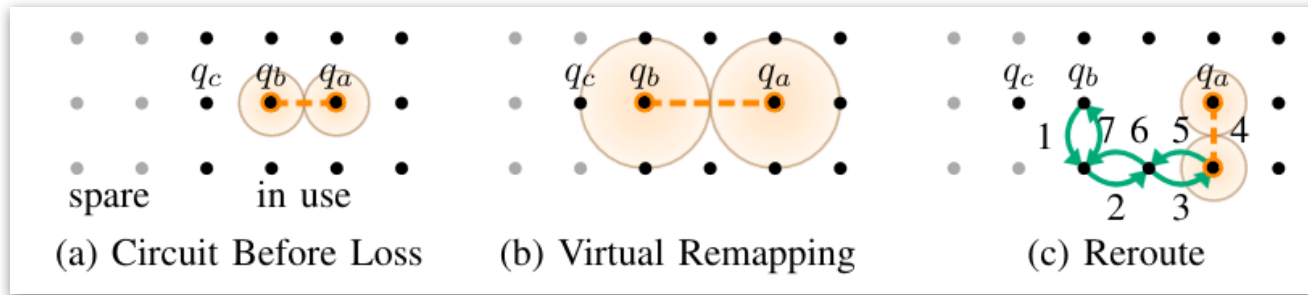


Image: Baker et al. (2021) *Exploiting Long-Distance Interactions and Tolerating Atom Loss in Neutral Atom Quantum Architectures*. ISCA '21.



# Qubit placement and routing

Techniques can use optimal solvers, heuristics, or a combination of both.

Example: SABRE (“SWAP-based BidiREctional heuristic search)

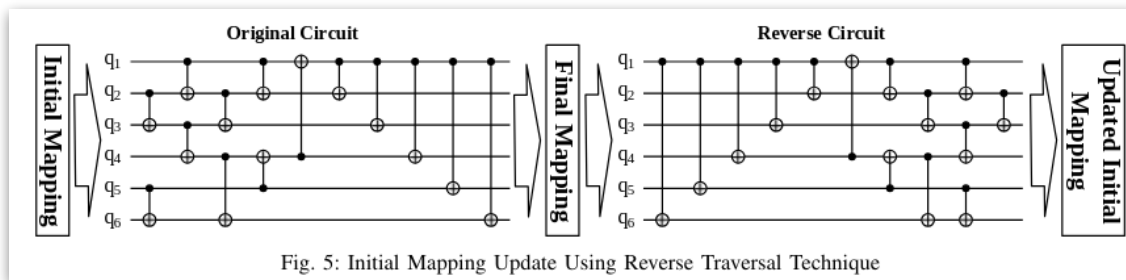


Fig. 5: Initial Mapping Update Using Reverse Traversal Technique

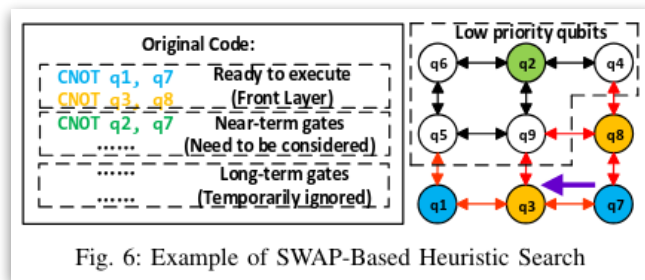
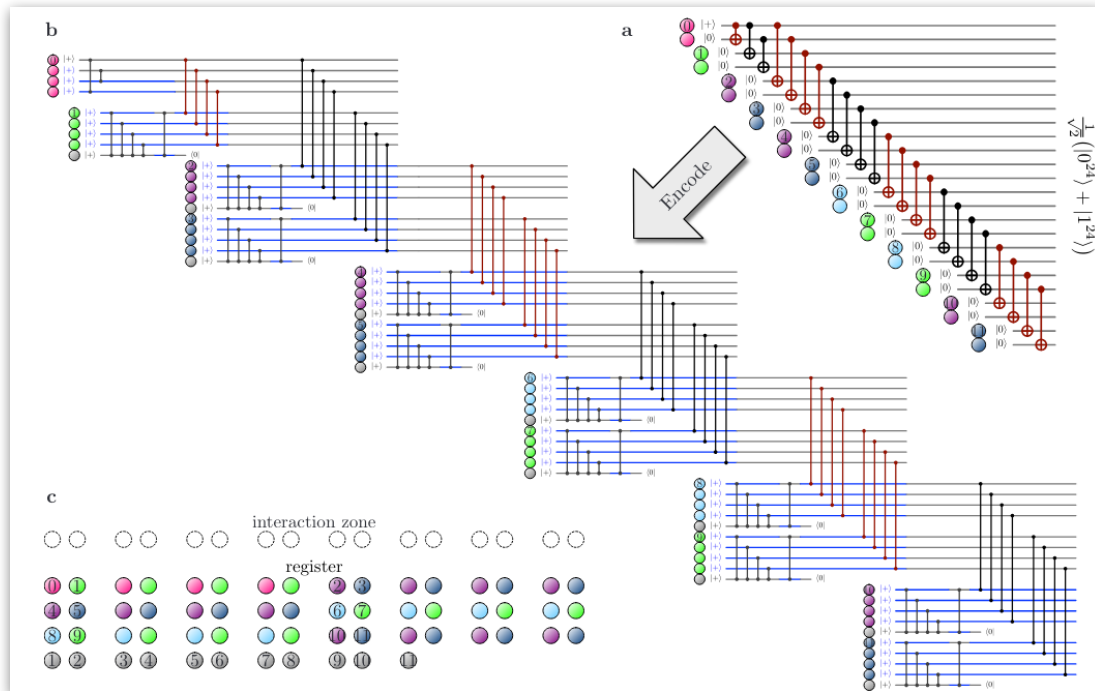


Fig. 6: Example of SWAP-Based Heuristic Search

G. Li, Y. Ding, and Y. Xie. 2019. Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices. In Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19). ACM, New York, NY, USA, 1001–1014.

# Gate scheduling



Recent demonstration of logical qubits on neutral atom machine: *move qubits in and out of “interaction zone” for 2-qubit gates*

(Microsoft + Atom computing + collaborators)

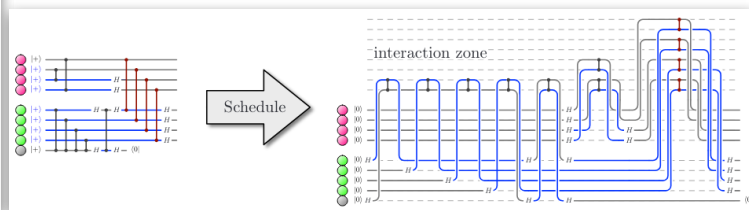
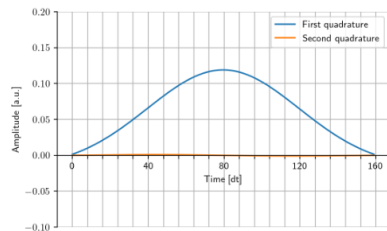


Image: Fig. 4 and 5, B. W. Reichardt et al., “Logical computation demonstrated with a neutral atom quantum processor”. arXiv:2411.11822 [quant-ph].

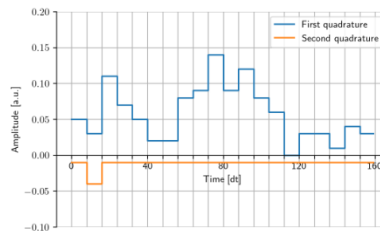
# Pulse design and optimization

Electromagnetic pulses *actually* implement the gates. Interesting problems:

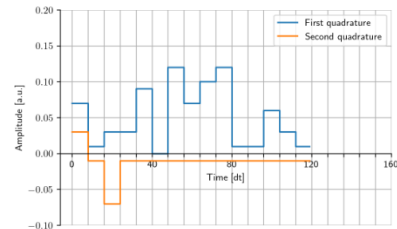
- optimizing duration/shape



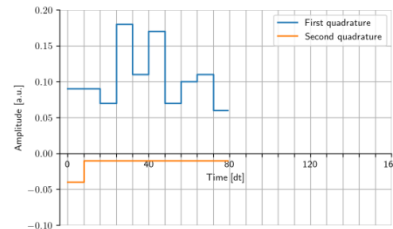
(a) DRAG  $X$  gate.



(b) Optimized  $X$  gate.



(c) Optimized  $X$  gate  $1.33\times$  faster.



(d) Optimized  $X$  gate  $2\times$  faster.

E. Wright and R. De Sousa, "Fast Quantum Gate Design with Deep Reinforcement Learning Using Real-Time Feedback on Readout Signals," in 2023 IEEE International Conference on Quantum Computing and Engineering (QCE), Bellevue, WA, USA, 2023, pp. 1295-1303

# Pulse design and optimization

Electromagnetic pulses *actually* implement the gates. Interesting problems:

- optimizing duration/shape
- scheduling constraints

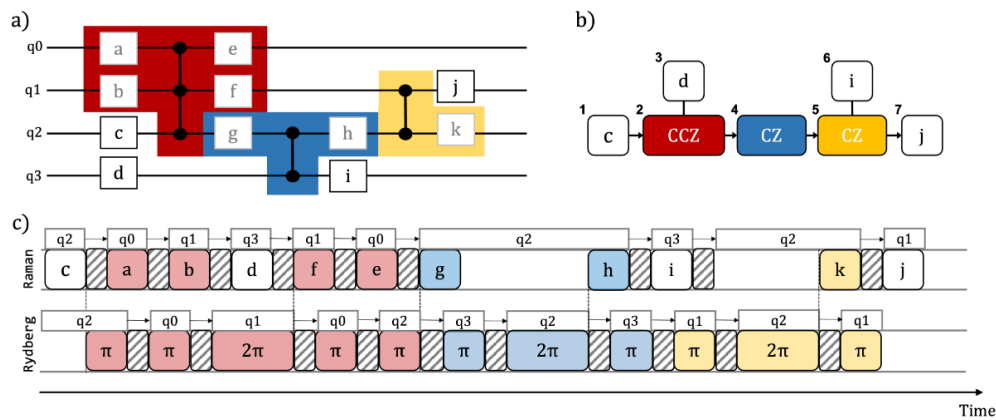


Image: R. B.-S. Tsai, H. Silvério, L. Henriët (2022) *Pulse-Level Scheduling of Quantum Circuits for Neutral-Atom Devices*. Phys. Rev. Applied 18 064035.

# Big problem: complexity and scale

Many tasks still require **human labour** and **expert experience**.

- How to better **automate** the process?
- How to **scale** to multi-QPU systems with 1000s of qubits?
- How to **verify** compiler output for circuits too large to simulate?

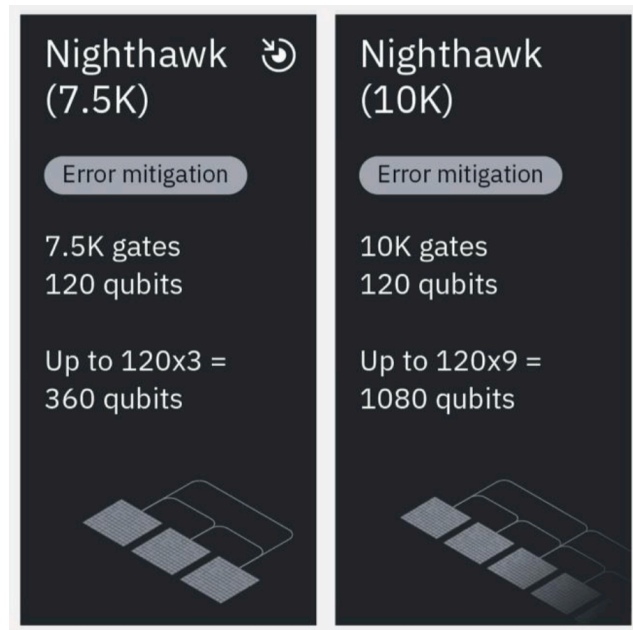


Image: IBM Quantum Roadmap (<https://www.ibm.com/downloads/documents/us-en/1443d5cda24021e4>)

## Bigger problem: error correction

We must revisit our assumptions and make some new ones...

- How good is the hardware?
- What code parameters do we need?
- What is the overhead of encoding?
- How do we apply logical operations?
- Which gates are expensive?

